SIAM J. SCI. COMPUT.
Vol. 18, No. 4, pp. 1014–1027, July 1997
© 1997 Society for Industrial and Applied Mathematics
004

MERGING COMPUTATIONAL ELEMENTS IN VORTEX SIMULATIONS*

LOUIS F. ROSSI†

Abstract. This paper analyzes the process of merging groups of many Gaussian basis functions into a single basis function in vortex simulations. Analysis of the equations governing this process yields fundamental parameters and uniform estimates of the merger-induced error. In a merging event, the uniform error bound depends only on relationships between nearby computational elements permitting fast and effective merging schemes. In this paper, one such algorithm is proposed and demonstrated.

Key words. vortex methods, Lagrangian algorithms, computational fluid dynamics

AMS subject classifications. 65D99, 65G99, 65M50, 65N50, 76C99, 76D99

PII. S1064827595285287

1. Introduction.

In scientific computation, the efficiency of a numerical scheme is often measured by the resources required to achieve a given accuracy. Usually, this accuracy is determined by the number of computational elements in a given simulation. For a gridded method, the computational resources required are an increasing function of the number of mesh points in the calculation. For vortex methods, the computational resources needed are an increasing function of the number of moving basis functions in the simulation. While vortex methods and other Lagrangian schemes can have advantages, a key disadvantage is that the moving basis functions can move toward one another so that they nearly overlap. Nearly overlapping computational elements can be expressed more simply as a single element in certain situations, reducing the computational load while maintaining the same accuracy. This paper examines the numerical merging of Gaussian computational elements to remove these extraneous degrees of freedom in vortex simulations.

Vortex methods simulate the two-dimensional, incompressible Navier–Stokes equations by approximating the vorticity field with a linear combination of basis functions. Each basis function or "vortex" has a position which moves with the local velocity at that point, a width and a circulation, and different investigators incorporate additional modifications to one of these three parameters to capture viscous diffusion [1, 5, 8, 6, 10, 11]. Though the vortex positions can be likened to mesh points for gridded calculations, the geometry of the vortex positions is not built into the data structure as is the case for gridded computations. Thus, it is possible and even likely for many flow simulations for several computational elements to move toward each other and eventually overlap. In this case, the dynamics of the movement and diffusion of the overlapping vortex elements may coincide to a point where a single element with appropriate position, circulation, and width could express the same information.

*Received by the editors April 28, 1995; accepted for publication (in revised form) November 17, 1995. This research was supported by US AF/OSF ASSERT grant 496092J0324DEF and by NSF grant DMS-9407660.

http://www.siam.org/journals/sisc/18-4/28528.html

†Department of Engineering Sciences and Applied Mathematics, Northwestern University, 2145 Sheridan Road, Evanston, IL 60208-3111. Current address: Department of Mathematical Sciences, University of Massachusetts Lowell, One University Avenue, Lowell, MA 01854 (rossil@woods.uml.edu).

This paper examines numerical merging rather than the physical merging of Gaussian vortex elements. The distinction and the relationship to one another is crucial to understanding and designing merging algorithms. Numerical merging is merely replacing several basis functions with a single basis function. Physical merging refers to the true vorticity dynamics when nearby patches or distributions of vorticity combine into a single element.

Previously, some investigators have used physical merging as a justification for numerical merging. For instance, Chorin removes parallel, opposite-signed vortex filaments to remove "hairpin vortices" in his three-dimensional vortex methods [3, 4]. In the case of two-dimensional merging and rapid axisymmetrization, there is a wealth of theory and numerical experimentation on this subject. Under very general conditions, nearby regions of same-signed vorticity will merge dynamically into axisymmetric monopoles [2, 9, 12, 13]. A more advanced form of merging algorithm may incorporate some of these results, but this paper will focus on numerical merging.

Numerical merger can be justified as a perturbation to the computational vorticity field. If a vortex method is stable, the introduction of small instantaneous errors in the computed vorticity field should not grow catastrophically as the system evolves. Indeed, the numerical merging of computational elements does not necessarily correspond to merging two distinct nearby regions of vorticity. In fact, as we shall see later in the paper, the computational elements must overlap considerably to control the induced error. Thus, a numerical merging event is more akin to perturbing a monopole than merging disjoint regions of vorticity.

This paper progresses from generality to specific implementations of the merging algorithm. Section 2 presents the basic formulation and notation for this problem. Section 3 develops two basic theorems describing the uniform error induced on a computational field when merging elements. In section 4, an explicit algorithm is developed from these uniform merger estimates. Finally, section 5 discusses the role of merging in two vortex method computations. In this case, the merging of vortex elements controls the problem size while introducing very small errors to the overall simulation.

2. Preliminaries. To begin, it is necessary to study a given configuration of basis functions and search for opportunities that will numerically merge computational elements in such a way as to disrupt the vorticity field by a small amount. The dynamics of the discrete system will not play a role in the analysis because it suffices to consider only the instantaneous positions, widths, and circulations of vortices and not how they came to be in the given configuration.

In a vortex method, the vorticity field, ω, is expressed as a linear combination of basis functions:

$$\omega(\vec{x}) = \sum_{i=1}^{N} \frac{\gamma_i}{4\pi\sigma_i^2} \exp\left(-\frac{|\vec{x}-\vec{x}_i|^2}{4\sigma_i^2}\right). \tag{1}$$

Each basis function has three parameters, $(\gamma_i, \vec{x}_i, \sigma_i)$, corresponding to its strength, position, and width, respectively. It is natural to assume that these basis function widths are bounded from below and above for a given simulation $l_m \leq \sigma_i \leq l_M$ for the following reasons. First, l_m would correspond to *maximum resolution.* For a fixed problem size, the lack of a lower bound would correspond to a numerical instability in the computational field because the basis functions would no longer overlap. Second, one would not want to merge elements into a very wide element because this would correspond to a loss of resolution. Thus, l_M is a *minimum resolution.*

Numerical merging will consist of replacing a collection of vortex elements with a single element. From the N computational elements, a subset $(\gamma_{i_j}, \vec{x}_{i_j}, \sigma_{i_j})$ for $1 \leq j \leq n$ shall be merged into a single element. The postmerger element will be labeled 0, and, for the remainder of this paper, the subindices i_j will be dropped because only the n elements to be merged are of importance to the analysis. (It is necessary to assume that all γ_i's $(1 \leq i \leq n)$ have the same sign for reasons to be discussed later in this section.) To understand the merging process, one must investigate the difference in the vorticity fields before and after the merging event:

$$e(\vec{x}) = \frac{\gamma_0}{4\pi\sigma_0^2}\left[\exp\left(-\frac{|\vec{x}|^2}{4\sigma_0^2}\right) - \sum_{i=1}^{n}\frac{\gamma_i}{\gamma_0}\frac{\sigma_0^2}{\sigma_i^2}\exp\left(-\frac{|\vec{x}-\vec{x}_i|^2}{4\sigma_i^2}\right)\right]. \tag{2}$$

The goal is to choose efficiently a collection of vortices together with the parameters $(\gamma_0, \vec{x}_0, \sigma_0)$ so that this error is small.

The process of bounding equation (2) begins with the assumption that numerical merging conserves zeroth, first, and second moments of vorticity. This effectively defines the numerical merging process because these constraints determine the strength, position, and width of the postmerger element:

$$\gamma_0 = \sum_{i=1}^{n}\gamma_i, \tag{3}$$

$$\vec{x}_0 = \sum_{i=1}^{n}\gamma_i\vec{x}_i, \tag{4}$$

$$4\gamma_0\sigma_0^2 = \sum_{i=1}^{n}\gamma_i(4\sigma_i^2 + |\vec{x}_i - \vec{x}_0|^2). \tag{5}$$

For convenience, all positions will be translated to the center of the mass of the original cluster of elements so that $\vec{x}_0 = 0$. Equation (5) constrains the merged blob width but does not play a key role in the error bound, as the others do. Therefore, it may be possible to use other conservation laws in place of the second moment. However, the merging algorithm described in section 4 will use this particular relationship.

In this context, an effective merging process would readily identify subsets of computational elements that can be merged using equations (3)–(5) while keeping $e(\vec{x})$ below a specified tolerance without altering l_m or l_M. Controlling $e(\vec{x})$ ensures that the computational field will only experience controllably small instantaneous disturbances. Maintaining $l_m \leq \sigma_i \leq l_M$ is necessary to maintain accuracy in the evolution of the computational field relative to the exact solution.

3. Uniform bounds on merger error. A variety of norms can be applied to $e(\vec{x})$ to determine how large an error will be induced by merging a group of vortex elements. One would expect merging to occur only for overlapping elements suggesting that globally merging errors will not accumulate in the same spatial region. Thus, the greatest pointwise error in the total vorticity field when merging is applied over the collection of vortices is likely to be comparable to the greatest pointwise error in an individual merging event. Indeed, many numerical computations using vortex merging have verified that this is the case. So, when studying a single merging event, the error induced by merging many elements into a single one is measured by the L_∞ norm though other norms could be applied. However, to use other norms such as L_2,

one would have to estimate how many individual merging events will occur over the entire spatial region.

To simplify $e(\vec{x})$, all the spatial vectors are nondimensionalized using

$$\frac{\vec{x}}{2\sigma_0} \to \vec{\hat{x}}. \tag{6}$$

Thus,

$$\begin{aligned} e(\vec{\hat{x}}) &= \frac{\gamma_0}{4\pi\sigma_0^2}\left\{\exp\left(-|\vec{\hat{x}}|^2\right) - \sum_{i=1}^{n}\frac{\gamma_i}{\gamma_0}\frac{\sigma_0^2}{\sigma_i^2}\exp\left[-|\vec{\hat{x}}-\vec{\hat{x}}_i|^2\left(\frac{\sigma_0^2}{\sigma_i^2}\right)\right]\right\} \\ &= \frac{\gamma_0}{4\pi\sigma_0^2}\sum_{i=1}^{n}\frac{\gamma_i}{\gamma_0}\left\{\exp\left(-|\vec{\hat{x}}|^2\right) - \frac{\sigma_0^2}{\sigma_i^2}\exp\left[-|\vec{\hat{x}}-\vec{\hat{x}}_i|^2\left(\frac{\sigma_0^2}{\sigma_i^2}\right)\right]\right\}. \end{aligned} \tag{7}$$

If one requires that all the γ's have the same sign, equation (7) is an arithmetic mean of the quantities

$$\exp\left(-|\vec{\hat{x}}|^2\right) - \frac{\sigma_0^2}{\sigma_i^2}\exp\left[-|\vec{\hat{x}}-\vec{\hat{x}}_i|^2\left(\frac{\sigma_0^2}{\sigma_i^2}\right)\right]. \tag{8}$$

That is to say,

$$e(\vec{\hat{x}}) \le \frac{|\gamma_0|}{4\pi\sigma_0^2}\max_{1\le i\le N}\left|\exp\left(-|\vec{\hat{x}}|^2\right) - \frac{\sigma_0^2}{\sigma_i^2}\exp\left[-|\vec{\hat{x}}-\vec{\hat{x}}_i|^2\left(\frac{\sigma_0^2}{\sigma_i^2}\right)\right]\right|. \tag{9}$$

Therefore, the induced error $e(\vec{\hat{x}})$ can be bounded in terms of the extreme values of (8). An interesting consequence is that bounding the total error induced by merging a collection of vortex elements is reduced to bounding the difference between a pair of Gaussians. Zero uniform error exists only if $\vec{\hat{x}} \to 0$ and $\frac{\sigma_0}{\sigma_i} \to 1$, the case where two identical Gaussians lie right on top of one another.

For fixed values of $\frac{\sigma_0}{\sigma_i}$ and $\vec{\hat{x}}_i$, the exponential decay of equation (8) assures the existence of a global extreme value, although its precise location may be difficult to determine analytically. If the parameters $\frac{\sigma_0}{\sigma_i}$ and $\vec{\hat{x}}_i$ are restricted to a compact set, the continuity of equation (8) guarantees the existence of a maximum value, M, on this set. Furthermore, the existence of this maximum yields a uniform estimate for equation (7). This is stated formally in the following theorem.

THEOREM 3.1 (merger error bound). *Suppose n computational elements merge into a single element with strength, position, and width defined by equations* (3), (4), *and* (5). *Furthermore, assume that all strengths have the same sign and that*

$$|\vec{\hat{x}}_i|^2 \le R$$

and

$$b_1 \le \left(\frac{\sigma_0}{\sigma_i}\right)^2 \le b_2$$

for each computational element. Then, the uniform-induced field error $\|e(\vec{\hat{x}})\|_\infty$ is no greater than

$$\frac{|\gamma_0|M}{4\pi\sigma_0^2},$$

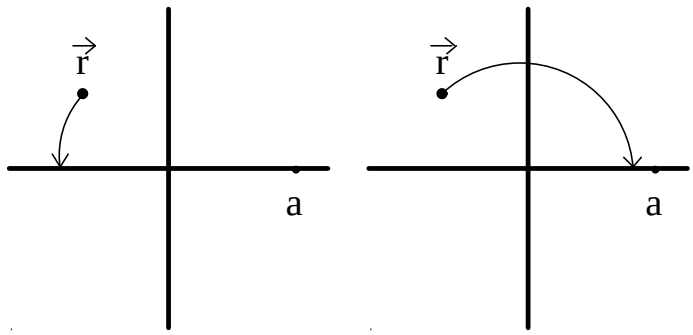


FIG. 1. *Increasing a "maximum" and decreasing a "minimum."*

where M is the maximum value of

$$\left|\exp\left(-|\vec{\widehat{x}}|^2\right) - \frac{\sigma_0^2}{\sigma_i^2}\exp\left[-|\vec{\widehat{x}} - \vec{\widehat{x}}_i|^2\left(\frac{\sigma_0^2}{\sigma_i^2}\right)\right]\right| \tag{10}$$

restricted to the compact set of parameter space described by R, b_1, and b_2.

To find M, it is not necessary to search the entire space $[0, R] \times [b_1, b_2]$. We shall see that these maximal values can only be attained on two corners, (R, b_1) and (R, b_2), of this rectangle in parameter space. Without loss of generality, the centroid of a given prefusion blob and the centroid of the postfusion blob both lie on the x axis. Then, equation (8) can be simplified as

$$f(\vec{x}, a, \beta) = \exp\left(-|\vec{\widehat{x}}|^2\right) - \beta\exp\left(-|\vec{\widehat{x}} - a\widehat{i}|^2\beta\right), \tag{11}$$

where $(a, \beta)\ \epsilon\ [0, R] \times [b_1, b_2]$. Finally, no generality is lost if $a > 0$.

First, one observes that any maximal value of this equation will occur on the x axis. If this were not the case, a maximum would be achieved at some point $\vec{r}$ away from the axis. A global maximum would necessarily be positive so the first term of equation (11) would have to be positive. But, one could rotate $\vec{r}$ onto the negative x axis (as shown in Fig. 1), reducing the latter negative term while the first term remains constant. Thus, $\vec{r}$ would not be a global maximum. A similar argument shows that f will achieve its minimal value on the x axis also. Thus, the maximal value of $|f(\vec{x}, a, \beta)|$ occurs on the x axis. Therefore, when considering extreme values, f will be considered a function of x rather than of $\vec{x}$. A corollary to this result is that maximum values occur when $x < 0$ and minimum values occur when $x > a$.

If there were a local extreme value for parameter values not on the boundary of $[0, R] \times [b_1, b_2]$, then

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial \beta} = 0.$$

However,

$$\frac{\partial f}{\partial a} = -2\beta^2(x-a)\exp[-(x-a)^2\beta], \tag{12}$$

$$\frac{\partial f}{\partial \beta} = [\beta(x-a)^2 - 1]\exp[-(x-a)^2\beta]. \tag{13}$$

Since β is close to unity, it is impossible to set equations (12) and (13) equal to zero simultaneously. Therefore, f can only be maximized on the boundary of the parameter space. Next, we shall see that this can be restricted to the two corners mentioned earlier.

To study the parameter dependencies, one must analyze the extreme values of f directly. A necessary condition is that $\frac{\partial f}{\partial x} = 0$, where

$$\frac{\partial f}{\partial x} = -2\beta^2(x-a)\exp[-(x-a)^2\beta] + 2x\exp(-x^2). \tag{14}$$

By substituting equation (13) into equation (14) at a maximal value with respect to x, one finds that

$$\frac{\partial f}{\partial a} = -2x\exp(-x^2). \tag{15}$$

If x is a maximum value of f, $\frac{\partial f}{\partial a}$ is a monotonically increasing function because, as observed earlier, x must be negative. However, if x is a minimum, then $\frac{\partial f}{\partial a}$ is monotonically decreasing because x is positive. Therefore, all extreme values are largest when $a = R$.

Similarly, substitution of equation (14) directly into f yields an expression for f at any local minimum, x_{min}, or maximum, x_{max}:

$$f(x_{min|max}, R, \beta) = \left[1 - \frac{x_{min|max}}{\beta(x_{min|max} - R)}\right] \exp\left(-x_{min|max}^2\right). \tag{16}$$

Any extreme value can be increased or decreased by moving β toward or away from the origin, proving the following theorem.

THEOREM 3.2 (explicit determination of merger error bound). *If*

$$\left(\frac{|\vec{x}_i|}{2\sigma_0}\right)^2 \leq R$$

and

$$b_1 \leq \left(\frac{\sigma_0}{\sigma_i}\right)^2 \leq b_2,$$

then the extreme values of the uniform merger error function

$$\exp\left(-|\vec{\hat{x}}|^2\right) - \frac{\sigma_0^2}{\sigma_i^2}\exp\left[-|\vec{\hat{x}} - \vec{\hat{x}}_i|^2\left(\frac{\sigma_0^2}{\sigma_i^2}\right)\right]$$

are achieved with parameter values of either (R, b_1) *or* (R, b_2). *Furthermore, this absolute maximum is attained when* $\vec{x}$ *is collinear with the origin and* $\vec{x}_i$.

The consequence of these theorems is that any number of same-signed computational elements can be fused together, provided they have sufficiently similar core widths and are in close proximity to each other relative to their core sizes. Furthermore, it is practical to tabulate extreme values of equation (11) because the entire parameter space no longer needs to be searched. Some values are provided in Table 1.

TABLE 1
Extreme values of equation (11) *tabulated for use in a merging algorithm.*

a	β	x_{min}	$-f_{min}$	x_{max}	f_{max}
10^{-3}	1.0	-1.41371	4.289×10^{-4}	1.41471	4.289×10^{-4}
10^{-3}	$1.0 + 1.0 \times 10^{-3}$	-2.2781	3.927×10^{-4}	0.4600	1.112×10^{-3}
10^{-3}	$1.0 - 1.0 \times 10^{-3}$	-2.4587	1.116×10^{-3}	2.2808	3.923×10^{-4}
10^{-3}	$1.0 + 1.0 \times 10^{-2}$	-2.7478	1.155×10^{-3}	0.0507	1.001×10^{-2}
10^{-3}	$1.0 - 1.0 \times 10^{-2}$	-2.7637	1.001×10^{-2}	0.0492	1.551×10^{-3}
10^{-2}	$1.0 + 1.0 \times 10^{-3}$	-1.5547	4.035×10^{-3}	1.2679	4.645×10^{-3}
10^{-2}	$1.0 - 1.0 \times 10^{-3}$	-1.1583	4.638×10^{-3}	1.5657	4.029×10^{-3}
10^{-2}	$1.0 + 1.0 \times 10^{-2}$	-2.2659	3.948×10^{-3}	0.4661	1.112×10^{-2}
10^{-2}	$1.0 - 1.0 \times 10^{-2}$	-0.4526	1.111×10^{-2}	2.2931	3.902×10^{-3}
10^{-1}	$1.0 + 1.0 \times 10^{-2}$	-1.5054	4.062×10^{-2}	1.3116	4.672×10^{-2}
10^{-1}	$1.0 - 1.0 \times 10^{-2}$	-1.2156	4.608×10^{-2}	1.6153	3.998×10^{-2}
10^{-1}	$1.0 + 1.0 \times 10^{-1}$	-2.1474	4.148×10^{-2}	0.5281	1.133×10^{-1}
10^{-1}	$1.0 - 1.0 \times 10^{-1}$	-0.3928	1.100×10^{-1}	2.4193	3.681×10^{-2}

4. Implementing a merging algorithm. The results of the previous section can be used to design an effective merging scheme for vortex simulations. The basic strategy is to determine a set of same-signed "candidate elements,"

$$\mathcal{C} = \{(\gamma_{i_1}, \vec{x}_{i_1}, \sigma_{i_1}), (\gamma_{i_2}, \vec{x}_{i_2}, \sigma_{i_2}), \ldots, (\gamma_{i_n}, \vec{x}_{i_n}, \sigma_{i_n})\}, \tag{17}$$

that can be merged into a single element without disturbing the total field beyond some specified tolerance ϵ. This strategy is applied $N-1$ times, attempting to create one candidate set, $\mathcal{C}$, for each element in the simulation except for the last element. The last element is excluded in this particular algorithm for reasons that will become clear later in the section. Thus, it is possible to have as many as $N-1$ merging events in a full pass or none at all, depending on the configuration vortex positions, widths, and circulations.

If one knows $\mathcal{C}$, it is possible to determine the postmerger element, $(\gamma_0, \vec{x}_0, \sigma_0)$, and therefore the merger error. If the error is less than the specified tolerance, one replaces all the elements of $\mathcal{C}$ with $(\gamma_0, \vec{x}_0, \sigma_0)$. Using Theorem 3.1, if

I. $\frac{|\vec{x}_i - \vec{x}_0|^2}{4\sigma_0^2} \leq R$,

II. $b_1 \leq \frac{\sigma_0^2}{\sigma_i^2} \leq b_2$,

III. $\frac{|\gamma_0| M}{4\pi\sigma_0^2} \leq \epsilon$,

where M depends on R, b_1, and b_2, then replacing $\mathcal{C}$ by $(\gamma_0, \vec{x}_0, \sigma_0)$ will induce a uniform error of no more than ϵ.

Since the selection of a candidate set requires computational resources, it is difficult to directly apply Theorem 3.1. Therefore, this particular merging implementation builds candidate sets that have a good chance of satisfying I, II, and III. Since any candidate set must be constructed one element at a time, conditions I, II, and III are not particularly helpful because they depend on $(\gamma_0, \vec{x}_0, \sigma_0)$. That is to say, it would be expensive to recompute equations (3)–(5) each time one is deciding whether or not to add a new element to $\mathcal{C}$. This implementation relies upon more restrictive hypotheses than I, II, and III to build a candidate set that has a good chance of identifying mergeable vortices.

To build the more restrictive hypotheses, one can use the merging description from equations (3)–(5) to make inferences about the postfusion vortex. From equation (5),

the lower bound on the width of the postfusion vortex is already satisfied, but the upper bound cannot be discerned before $\mathcal{C}$ is formed. To build equation (5), it is clear that $\sigma_0 \geq l_m$. Also, from equation (4), $\vec{x}_0$ must lie in the convex hull of $\{\vec{x}_{i_k}\}$. Hence, if

$$|\vec{x}_j - \vec{x}_i|^2 \leq \frac{D}{4}$$

for all j and one particular i in $\mathcal{C}$, then from the triangle inequality

$$|\vec{x}_j - \vec{x}_0|^2 \leq D.$$

Using this information, the following more restrictive conditions help select candidate vortices:

I′. $|\vec{x}_i - \vec{x}_j|^2 \leq Rl_m^2$ for all j and one particular i in $\mathcal{C}$.

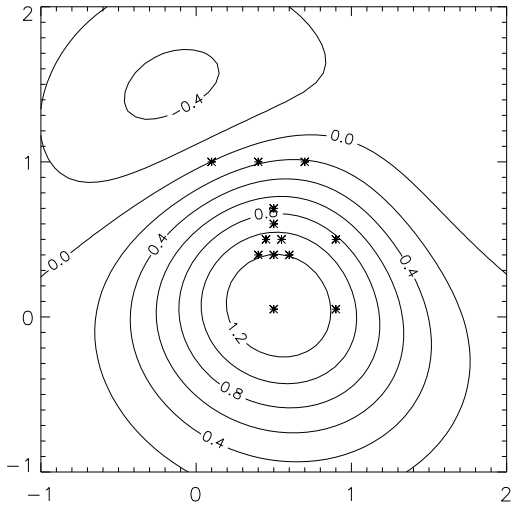
III′. $|\gamma_0| \leq \epsilon \frac{4\pi l_m^2}{M}$.

With this assumption, I′ and III′ imply I and III, respectively. Nothing is known about whether or not II is satisfied or $\sigma_0 \leq l_M$. The advantage to using I′ and III′ is that one need not compute σ_0 as each candidate set, $\mathcal{C}$, is constructed. Determining σ_0 is relatively expensive because adding a new element to $\mathcal{C}$ will most likely cause the center of vorticity to shift, requiring a re-evaluation of the entire sum in equation (5). However, determining γ_0 after adding a new element to $\mathcal{C}$ only requires an addition operation.

Finally, this procedure is repeated $N-1$ times. On the first pass, a candidate set is created using the first vortex element, $i = 1$, and any others, $j > 1$, that satisfy I′ and III′. If this set satisfies II, all the elements are merged into element $i = 1$. On the second pass, a new candidate set is constructed consisting of the second computational element (assuming that it has not already been combined into $i = 1$) and any others, $j > 2$, that satisfy I′ and III′. This process is repeated until $i = N - 1$. It is not necessary to create a set starting with the Nth element because there are no more elements $j > N$ to test against the hypotheses. This procedure ensures that every computational element can only be involved in at most one merging event.

The scheme described above will build $N-1$ candidate sets, $\mathcal{C}$, and can be added to a vortex code during the velocity computation. During the velocity calculation, the pairwise quantity $\vec{x}_i - \vec{x}_j$ at each position $\vec{x}_i$ can also be used to check the hypotheses for merging. What follows is an outline of this merging algorithm.

1. The error tolerance ϵ, and therefore M, is specified. From M, one can determine R, b_1, and b_2 from tables such as Table 1.
2. Loop on i from 1 to N.
3. Set $\mathcal{C} = \{(\gamma_i, \vec{x}_i, \sigma_i)\}$.
4. Loop on j from 1 to N to compute $d_{ij} = \vec{x}_i - \vec{x}_j$ for each j. At this point, one can determine the jth elements' contribution to the velocity field at $\vec{x}_i$.
5. Element j is added to $\mathcal{C}$ if all the following conditions hold.
   a. $i < j$. This will ensure that element i will participate in a maximum of one merging event and so guarantee that error will not accumulate through multiple sequential events.
   b. γ_i has the same sign as γ_j.
   c. Adding element j to $\mathcal{C}$ will not cause the total strength in $\mathcal{C}$ to exceed $\epsilon \frac{4\pi l_m^2}{M}$.
   d. $|d_{ij}|^2 \leq Rl_m^2$.
6. End loop j.



i	γ_i	$\vec{x}_i$	σ_i
1	0.1	(0.50, 0.05)	0.49
2	0.4	(0.90, 0.05)	0.46
3	0.2	(0.40, 0.40)	0.47
4	0.1	(0.60, 0.40)	0.47
5	0.1	(0.50, 0.40)	0.49
6	0.2	(0.45, 0.50)	0.45
7	0.1	(0.55, 0.50)	0.47
8	-0.7	(0.90, 0.50)	0.48
9	0.1	(0.50, 0.60)	0.48
10	0.2	(0.50, 0.70)	0.46
11	-0.5	(0.10, 1.00)	0.49
12	-0.1	(0.40, 1.00)	0.47
13	0.1	(0.70, 1.00)	0.48

FIG. 2. *A sample collection of basis functions for a Lagrangian simulation before merging. Left: The particle positions superposed on the computational field. Right: The particle parameters generating the computational field.*

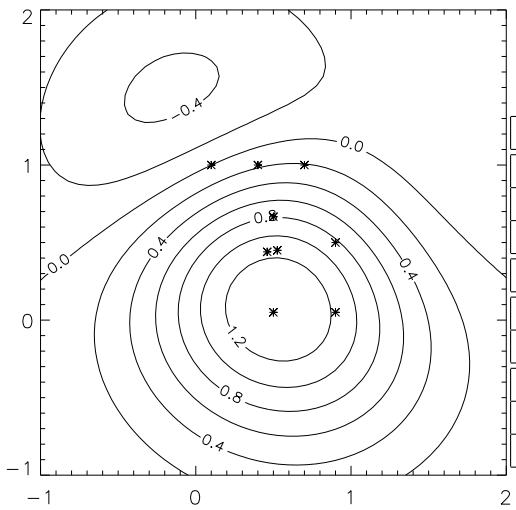
7. If $|\mathcal{C}| > 1$, compute $(\gamma_0,\vec{x}_0,\sigma_0)$. If $\sigma_0 < l_M$ and condition II is satisfied for each element in $\mathcal{C}$, replace element i with postmerger element 0 and remove all other elements in $\mathcal{C}$ from the simulation. Note that all elements in $\mathcal{C}$ automatically satisfy I and III, as discussed earlier.

8. End loop i.

This algorithm has a variety of qualities which must be considered before it is implemented. First, the choice of R, b_1, and b_2 will affect the performance of this algorithm, and a wide variety of these values will yield the same M. Features which affect the choice of these parameters would be the numerical parameters such as N or l as well as the flow to be simulated. The algorithm is also index dependent. The algorithm first attempts to build $\mathcal{C}$ by considering element 1's relationship with individual elements 2 through N. Then it tries to build $\mathcal{C}$ considering 2's relationship with elements 3 through N, and so on. Also, a lower-indexed element has a greater chance of joining a candidate set than a higher-indexed element because γ_0 grows as $\mathcal{C}$ is constructed. Also, although uniform field errors will not accumulate spatially, they will accumulate in time. Therefore, it is recommended that the merging algorithm be applied on a time scale, ΔT_{merge}, distinct from other numerical parameters so that the worst case field error induced by fusion is $\epsilon \frac{T}{\Delta T_{\text{merge}}}$.

EXAMPLE 1 (application of the merging algorithm). *A collection of computational elements is involved in a simulation. While evaluating pairwise interactions, the merging algorithm will check to see if any of them can be combined into a single element, saving computational time and effort. The elements together with an image of the induced field are provided in Fig.* 2.

The merging parameters $\epsilon = 0.2$, $R = 10^{-1}$, $b_1 = 1.0 - 10^{-1}$, *and* $b_2 = 1.0 + 10^{-1}$ *are chosen with the help of the tabulated values in Table* 1. *This establishes a value of* $M = 1.133 \times 10^{-1}$. *The simulation parameters are* $N = 13$, $l_M = 0.5$, *and* $l_m = 0.45$. *In this case,* $Rl_m^2 = 0.02025$ *and* $\frac{4\pi l_m^2}{M} = 22.459753$.



i	γ_i	$\vec{x}_i$	σ_i
1	0.1	(0.50, 0.05)	0.49
2	0.4	(0.90, 0.05)	0.46
3	0.5	(0.46, 0.44)	0.464...
5	0.2	(0.525, 0.45)	0.481...
8	-0.7	(0.90, 0.50)	0.48
9	0.3	(0.50, 0.667...)	0.467...
11	-0.5	(0.10, 1.00)	0.49
12	-0.1	(0.40, 1.00)	0.47
13	0.1	(0.70, 1.00)	0.48

FIG. 3. *A sample collection of basis functions for a Lagrangian simulation after merging. Left: The particle positions superposed on the computational field. Right: The particle parameters generating the computational field.*

During the first pass through the outer loop of the algorithm ($i = 1$), $\mathcal{C}$ *only contains element* 1 *because all other elements are too far away. That is,* $|\vec{x}_1 - \vec{x}_j|^2 > Rl_m^2$ *for all* $j > 1$. *The same thing happens on the second sweep at* $i = 2$. *On the third sweep* ($i = 3$) *elements* 4 *and* 6 *are added to* $\mathcal{C}$ *because they satisfy requirements* (a)–(d) *in item* 5 *of the algorithm. To test to see whether the candidates satisfy all requirements of the fusion theorems, the postfusion vortex is computed to be* $(\gamma_0, \vec{x}_0, \sigma_0) = (0.5, (0.46, 0.44), 0.464209)$. *All elements in* $\mathcal{C}$ *satisfy condition* II, *so element* 3 *is replaced by the postfusion element and elements* 4 *and* 6 *are deleted. By the same procedure, elements* 5 *and* 7 *are fused, as are* 9 *and* 10. *No other fusion events occur with these parameters. The final configuration is shown in Fig.* 3. *The problem size has been reduced from* 13 *to* 9, *reducing the number of pairwise interactions to less than half of those required by the original configuration.*

This example demonstrates how redundant information is related to the parameters controlling the error bound. The characteristic width of the individual elements is close to the characteristic length of the vortex structure itself. For instance, elements 3, 4, and 6 are quite close to each other relative to the average width of the basis functions. The merging scheme finds those elements that closely overlap relative to this characteristic width and merges them together. When this overlap does not exist, merging does not and should not occur without further consideration of physical vortex merger. Although the error tolerance ϵ is purposely chosen to be large to demonstrate merging with a small number of elements, the tolerance can be decreased to preserve accuracy while still removing redundant elements in larger simulations.

5. Applications to vortex methods. To demonstrate the overall effectiveness of this simple merging procedure, the algorithm is coupled to a vortex method simulation of a two-dimensional, incompressible flow. The vortex method, called the corrected core spreading vortex method (CCSVM), regularly adds elements to maintain accuracy as vorticity diffuses across streamlines [10]. Given an initial distribution of

vortex elements, CCSVM integrates the vorticity field in time by solving the following system of ODEs:

$$\dot{\vec{x}}_i = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \sum_{j=1}^{N} \frac{\gamma_i}{2\pi} \frac{\vec{x}_i - \vec{x}_j}{|\vec{x}_i - \vec{x}_j|^2} \left[1 - \exp\left(-\frac{|\vec{x}_i - \vec{x}_j|^2}{4\sigma_i^2} \right) \right], \tag{18}$$

$$\dot{\sigma}_i^2 = \nu. \tag{19}$$

Since the vortex elements spread as they convect, a correction is required to maintain resolution and accuracy.

This correction, called adaptive spatial refinement, replaces elements that have grown larger than a specified width, l, with four thinner elements of size $\alpha\sigma$, where $\alpha \in [0, 1]$. The new elements have one-fourth strength or circulation of the original wider element and are positioned symmetrically a distance r around the position of the original, where $r = 2\sigma\sqrt{1-\alpha^2}$. With this choice of r, the zeroth, first, and second moments of the vorticity field are conserved exactly in the refinement process. However, refinement disturbs the computational field slightly by introducing a small error controlled by α. As α approaches unity, the error goes to zero but the problem size grows infinitely large. Similarly, l controls the minimum resolution of the simulation because features smaller than l cannot be captured with the basis functions in equation (1). In terms of merging parameters, $l_m = \alpha l$ and $l_M = l$. Together, α and l are the two numerical parameters controlling the accuracy of the simulation.

Two experiments instantiate the role of merging in controlling the problem size of particle simulations. The first involves a radially symmetric patch of vorticity which steadily spreads under the action of viscosity. In this case, the vorticity field is not changed by convective action and the problem size will grow unreasonably large without some way to reduce unnecessary degrees of freedom. The second involves two patches of same-signed vorticity. In this case, convection is an important effect in many regions of the flow. Thus, merging is only required in particular localized regions.

For the case of a single radially symmetric patch of vorticity, the initial condition is a single vortex element ($\gamma = 5.0$, $\sigma = 0.2$). The maximum resolution l is 0.25 since this flow does not have small-scale features. The refinement parameter α is allowed to vary over a range of values from 0.7 up to 0.9. As α grows, the accuracy and problem increase as expected. However, the problem size grows much too fast as α grows toward unity. At $\alpha = 0.9$, the problem size grows to $N = 4^7 = 16384$ by the end of the simulation, requiring over six hours of CPU on a Sparc 2. By incorporating the merging procedure into the code, this performance improved substantially. The program checked for possible clusters of vortex elements to merge at regular times, $t = 0.1$, 0.2, etc., allowing a maximum pointwise field error of 0.01 at any isolated merger event. With this method, the final problem size was $N = 4050$, and the simulation ran in less than 20 minutes. In Fig. 4, one can see how element merging effectively reduces the problem size in areas where the vorticity field is either weak or unnecessarily dense. The simulations with and without merging produce almost identical results in that the uniform difference between vorticity fields is less than 0.002 (see Fig. 5) and the velocity fields are indistinguishable. In fact, this simulation is more accurate than the $\alpha = 0.85$ simulation even though it uses fewer elements. More aggressive merger tolerances would produce greater performance with less accuracy.

Convection plays a greater role in the simulation of two merging patches of vorticity. Originally, the entire field is approximated with only two computational elements

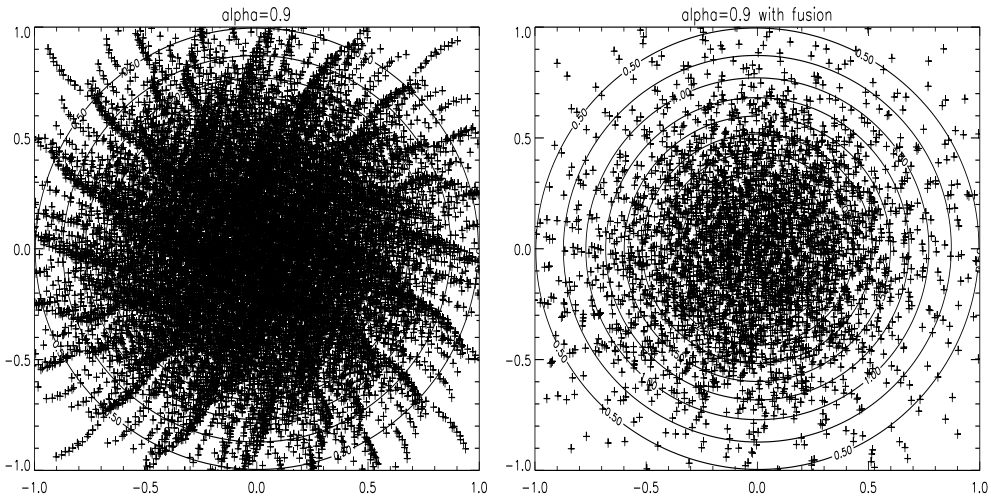


FIG. 4. *Vorticity field with element positions at $t = 1.0$ for $\alpha = 0.9$ without merging (left) and with merging (right).*

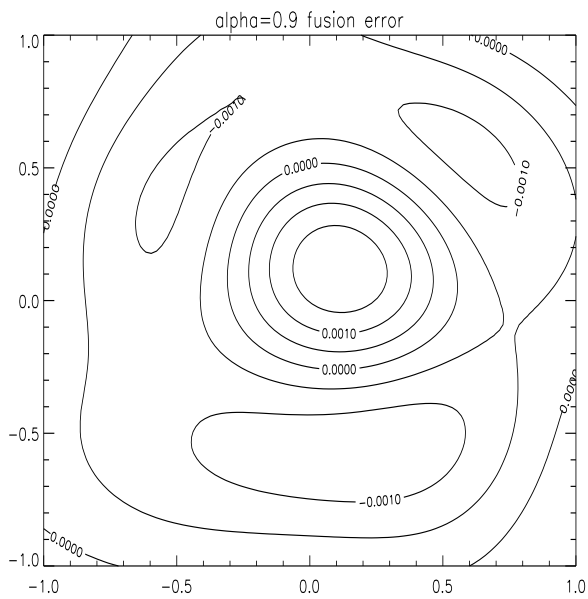


FIG. 5. *Symmetric patch of vorticity: Distribution of merger error at $\alpha = 0.9$.*

($\gamma = 5.0$, $\sigma = 0.2$) separated by a distance of 1.6 along the x axis. The viscosity is 0.02, so the Reynolds number is 5000. The minimum resolution l is 0.2 and T is 2.5. As the simulation evolves, the two regions of vorticity diffuse and rotate about one another, eventually coalescing into a single patch of vorticity. Unlike the radially symmetric patch, this flow is unsteady even in a frame rotating with the centroids of the two patches because these distributions are being sheared apart. More details of this particular experiment using CCSVM can be found in [10]. For the same merging error tolerance used above, the problem size of this experiment with $\alpha = 0.85$ dropped from $N = 2048$ without merging to $N = 698$ with merging. In Fig. 6, one readily sees that

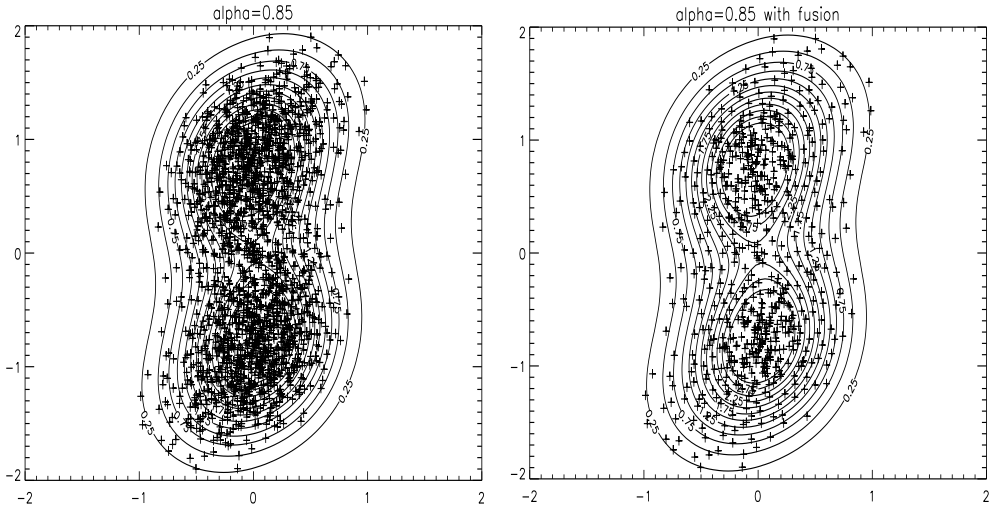


FIG. 6. *Two patches of vorticity. Vorticity field with element positions at* $t = 2.5$ *for* $\alpha = 0.85$ *without merging (left) and with merging (right).*

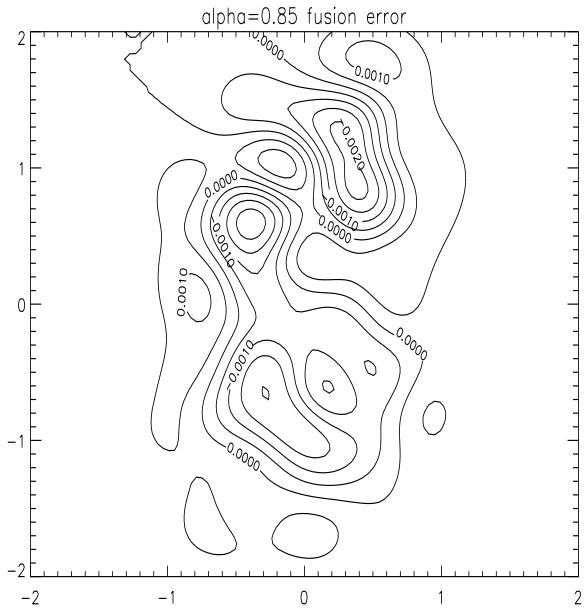


FIG. 7. *Distribution of merger error for two patches of vorticity at* $\alpha = 0.85$ *and* $t = 2.5$. *The relative error is less than* 0.06%.

though the problem size has been greatly reduced through the merging algorithm, the vorticity fields are indistinguishable from one another.

Examination of the difference between these two fields in Fig. 7 indicates that the greater absolute errors occur in the regions of highest particle density, as one would expect. The index dependence in the merging algorithm is responsible for the asymmetry in the field error. However, the zeroth, first, and second moments of this field error are zero to a precision far below that of the flow itself.

6. Conclusion and discussion. Merging basis functions is a practical and effective way to reduce extraneous degrees of freedom from Lagrangian particle simulations. This is not to imply that it will always be easy to find extraneous degrees of freedom to remove. However, for those flows that tend to cause basis functions to overlap or accumulate over time, a merging algorithm can decrease the computational resources required for the simulation.

This paper studies one particular basis function, the Gaussian, to determine uniform error estimates for merger events. A technique for fusing together computational elements arises from these uniform estimates, and this technique is demonstrated to be effective in nontrivial test problems. An obvious extension of this work would examine other basis function shapes and determine what error estimates are possible.

The merging algorithm demonstrated in this paper is not without its limitations. For one, the work is only applicable to Gaussian basis functions. Their particular functional form is used in obtaining uniform error estimates, but there are many other forms of basis functions in common use. Also, this particular implementation is designed to augment simulations involving direct pairwise interactions between computational elements. However, since merging really only requires communication between nearby elements, a merging algorithm for fast multipole algorithms such as that of Greengard and Rokhlin would be a logical extension of this work [7]. Finally, the merging algorithm presented here is not parallelizable. Since vortex computations are parallelizable, a simultaneous merging algorithm would have obvious advantages.

Finally, though this paper focuses on vortex methods, the merging techniques could be applied to other Lagrangian methods. For instance, the diffusion of a passive scalar or two-dimensional magnetohydrodynamic system can be solved numerically with moving basis functions. In this paper, the actual dynamics of the method play a very small role in the merging algorithm. Rather, the merging relies on instantaneous analysis of the basis functions and the stability of the numerical method. Thus, any stable Lagrangian method might benefit from a merging algorithm to remove extraneous degrees of freedom.

REFERENCES

[1] C. Anderson and C. Greengard, *On vortex methods*, SIAM J. Numer. Anal., 22 (1985), pp. 413–440.

[2] A. J. Bernoff and J. F. Lingevitch, *Rapid relaxation of an axisymmetric vortex*, Phys. Fluids, 6 (1994), pp. 3717–3723.

[3] A. J. Chorin, *Hairpin removal in vortex interactions*, J. Comput. Phys., 91 (1990), pp. 1–21.

[4] A. J. Chorin, *Hairpin removal in vortex interactions* II, J. Comput. Phys., 107 (1993), pp. 1–9.

[5] A. J. Chorin, *Numerical study of slightly viscous flow*, J. Fluid Mech., 57 (1973), pp. 785–796.

[6] P. Degond and S. Mas-Gallic, *The weighted particle method for convection-diffusion equations. Part* I: *The case of an isotropic viscosity*, Math. Comp., submitted.

[7] L. Greengard and V. Rokhlin, *A fast algorithm for particle simulations*, J. Comput. Phys., 73 (1987), pp. 325–348.

[8] A. Leonard, *Vortex methods for flow simulation*, J. Comput. Phys., 37 (1980), pp. 289–335.

[9] M. V. Melander, N. J. Zabusky, and J. C. McWilliams, *Symmetric vortex merger in two dimensions: Causes and conditions*, J. Fluid Mech., 195 (1988), pp. 303–340.

[10] L. F. Rossi, *Resurrecting core spreading vortex methods: A new scheme that is both deterministic and convergent*, SIAM J. Sci. Comput., 17 (1996), pp. 370–397.

[11] J. A. Sethian, *A brief overview of vortex methods*, in Vortex Methods and Vortex Motion, K. Gustafson and J. A. Sethian, eds., SIAM Frontiers in Applied Mathematics, SIAM, Philadelphia, PA, 1990.

[12] L. Ting and R. Klein, *Viscous vortical flows*, Lecture Notes in Phys., Vol. 374, Springer-Verlag, Berlin, 1991.

[13] J. B. Weiss and J. C. McWilliams, *Temporal scaling behavior of decaying two-dimensional turbulence*, Phys. Fluids A, 5 (1993), pp. 608–621.